

TRABALHO FINAL DE SISTEMAS OPERACIONAIS

UNIVERSIDADE FEEVALE

ALUNOS:

ARTHUR DUARTE FERNANDES

FERNANDO GOETZ MUELLER

GUILHERME MANENTE CRUZ BERLITZ

LAÍSSA DORNELLES SALLES

MATHEUS FELIPE MÜLLER

NICOLAS DALOSTO JAHNO

Professor:

BLANDA HELENA DE MELLO

NOVO HAMBURGO/RS

2026/01



SUMÁRIO

Introdução.....	3
Descrição do algoritmo.....	3
Premissas.....	3
Estruturas de dados.....	4
Fluxo de execução.....	5
Execução.....	6
Docker.....	8
Saída do simulador.....	9
Interpretação.....	11
Conclusão.....	11



Introdução

O objetivo deste trabalho foi desenvolver um simulador de escalonamento de processos utilizando o algoritmo **Round Robin com Feedback**, integrando operações de entrada/saída (I/O). O projeto se relaciona diretamente com os conceitos de **Sistemas Operacionais**, pois aborda gerenciamento de processos, filas de prioridade, preempção, bloqueio por dispositivos e métricas de desempenho — elementos centrais no funcionamento de um SO real.

Descrição do algoritmo

O algoritmo implementado é o Round Robin com Feedback:

- Processos novos entram na fila de alta prioridade.
- Cada fila possui quantum diferente: 2 ticks para alta e 4 ticks para baixa.
- Se um processo consome todo o quantum sem finalizar ou solicitar I/O, ele é rebaixado para a fila de baixa prioridade.
- Processos que retornam de I/O podem voltar à fila alta ou baixa, dependendo do dispositivo.
- A CPU sempre escolhe os processos da fila de maior prioridade disponível.

Premissas

As principais premissas adotadas no simulador são:

- **Quantum da fila ALTA:** 2 ticks.
- **Quantum da fila BAIXA:** 4 ticks.
- **Processos novos:** todos os processos iniciam na fila ALTA.
- **Dispositivos de I/O:**
 - **Disco:** duração de 4 ticks, com retorno à fila BAIXA.
 - **Fita:** duração de 6 ticks, com retorno à fila ALTA.



- **Impressora:** duração de 3 ticks, com retorno à fila ALTA.
- **Quantidade de processos:** não há limite fixo; o simulador aceita os processos definidos no arquivo input/processes.csv.
- **Tempo de CPU:** definido diretamente no CSV pela coluna TIME_REMAINING. No exemplo utilizado, os processos possuem entre 2 e 8 unidades de CPU.
- **Semente aleatória:** não utilizada. Os processos são definidos integralmente via CSV, garantindo reprodutibilidade.
- **Critério de finalização:** a simulação encerra quando as filas de CPU, as filas de I/O e a CPU estão livres, não havendo mais processos prontos, bloqueados ou em execução.

Estruturas de dados

- **PCB (Process Control Block):** classe implementada em pcb.py, responsável por armazenar o estado completo de cada processo durante a simulação. Os campos apresentados na tabela correspondem diretamente aos atributos utilizados para controle de identificação, estado, prioridade, tempo restante de CPU, dispositivo de I/O e métricas de execução.

CAMPO	VALORES POSSÍVEIS	FINALIDADE
pid	Inteiro	Identificador único do processo
ppid	0 ou PID do pai	Identificador do processo pai
status	NEW, READY, RUNNING, WAITING, TERMINATED	Estado atual do processo
priority	HIGH, LOW	Fila em que o processo está inserido



time_remaining	Inteiro (ticks)	Unidades de CPU ainda necessárias
io_device	NENHUM, DISCO, FITA, IMPRESSORA	Dispositivo de I/O associado
io_wait_time	Inteiro (ticks)	Contador regressivo do I/O em andamento
waiting_time	Inteiro (ticks)	Tempo acumulado esperando em fila de CPU
preemptions	Inteiro	Quantidade de preempção sofrida
io_events	Inteiro	Total de operações de I/O realizadas
turnaround	Inteiro (calculado)	finish_time – arrival_time

- **Filas de CPU:** duas listas (queue_high e queue_low) para processos prontos.
- **IOManager:** gerencia três filas independentes (disco, fita, impressora), controlando tempos de espera e reintegração dos processos ao escalonador.

Fluxo de execução

1. **Criação:** processos são lidos do CSV e inseridos na fila ALTA.
2. **Execução:** a CPU seleciona o próximo processo conforme prioridade.
3. **Preempção:** se o quantum expira, o processo é rebaixado ou rotacionado na fila BAIXA.
4. **Bloqueio por I/O:** se o processo solicita I/O, é enviado ao dispositivo correspondente e aguarda o tempo definido.
5. **Finalização:** quando o tempo restante chega a zero, o processo é marcado como TERMINATED e suas métricas são registradas.



6. **A cada tick, o simulador executa as etapas na seguinte ordem:**

1. Incrementa o tempo de espera de todos os processos nas filas de CPU.
2. Avança as filas de I/O e reintegra processos que concluíram o I/O.
3. Realiza troca de contexto se a CPU estiver livre (fila alta tem prioridade absoluta sobre a fila baixa).
4. Executa um tick de CPU, tratando os eventos nesta ordem de prioridade: (A) CPU ociosa, (B) processo finalizado, (C) bloqueio por I/O, (D) preempção por quantum expirado.

Essa ordem garante que processos liberados do I/O já estejam disponíveis para escalonamento no mesmo tick em que concluem a operação.

Execução

O projeto foi desenvolvido em **Python 3.12**, sem dependências externas. Para executar:

```
# A partir da raiz do projeto  
python src/main.py
```

Além da execução local com Python, o projeto também pode ser executado por Docker, conforme descrito no próximo tópico.

O simulador lê input/processes.csv e gera o log em output/execution_log.txt.

O arquivo CSV utiliza quatro colunas:



COLUNA	TIPO	DESCRIÇÃO
pid	Inteiro	IDENTIFICADOR ÚNICO DO PROCESSO
ppid	Inteiro	PID do processo pai (0 = processo raiz)
time_remaining	Inteiro	Unidades de CPU necessárias para finalizar
io_type	Inteiro	Dispositivo: 0=nenhum 1=disco 2=fita 3=impressora

Exemplo utilizado na simulação (input/processes.csv):

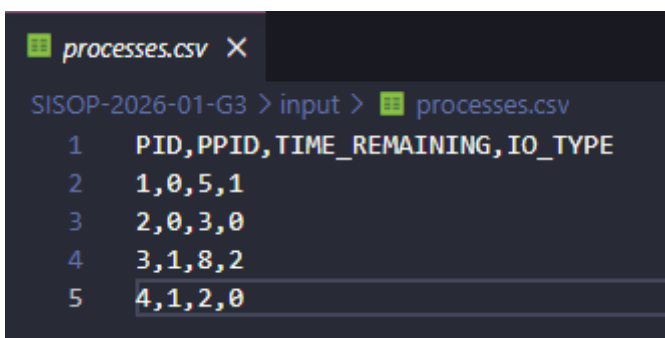
PID, PPID, TIME_REMAINING, IO_TYPE

1, 0, 5, 1

2, 0, 3, 0

3, 1, 8, 2

4, 1, 2, 0



```
processes.csv X
SISOP-2026-01-G3 > input > processes.csv
1 PID,PPID,TIME_REMAINING,IO_TYPE
2 1,0,5,1
3 2,0,3,0
4 3,1,8,2
5 4,1,2,0
```



Docker

Um Dockerfile foi incluído no projeto para permitir a execução do simulador em ambiente isolado, sem necessidade de configurar manualmente o Python na máquina local. Dessa forma, o projeto pode ser executado de maneira padronizada em diferentes ambientes, desde que o Docker esteja instalado.

Para construir a imagem Docker, a partir da raiz do projeto, utiliza-se o comando:

```
docker build -t so-escalador-g3 .
```

Esse comando cria uma imagem chamada **so-escalador-g3**, contendo o código-fonte do simulador e os arquivos necessários para sua execução.

Após a construção da imagem, o simulador pode ser executado com:

```
docker run --rm so-escalador-g3
```

O parâmetro **--rm** remove automaticamente o container após a finalização da execução, evitando a permanência de containers parados no ambiente.

Além da execução simples, também é possível executar o container com montagem de volume para persistir o arquivo de log na máquina local. Essa forma é útil porque o log gerado dentro do container pode ser perdido quando o container é removido ao final da execução. Com o volume, a pasta output do projeto local é vinculada à pasta /app/output dentro do container, garantindo que o arquivo execution_log.txt fique disponível no projeto mesmo após o encerramento do container.

No Linux, macOS ou WSL2:

```
docker run --rm -v "$(pwd)/output:/app/output" so-escalador-g3
```

No Windows PowerShell:

```
docker run --rm -v "${PWD}/output:/app/output" so-escalador-g3
```



Assim, o simulador continua lendo os processos definidos em input/processes.csv e o log da execução é salvo em output/execution_log.txt.

Saída do simulador

A imagem a seguir apresenta uma **execução real do simulador** utilizando o arquivo input/processes.csv de exemplo. O log mostra a criação dos processos, trocas de contexto, solicitações e retornos de I/O, preempções, finalizações e métricas finais.



```

PS C:\Users\Laissa\OneDrive - BEG\Área de Trabalho\clone\SISOP-2026-01-G3> docker run --rm -v "${PWD}/output:/app/output" so-escalonador-g3
[t=000] Processo criado - Fila ALTA: [PID:1 | Status:READY | Prioridade:HIGH | TempoRestante:5 | I/O:DISCO]
[t=000] Processo criado - Fila ALTA: [PID:2 | Status:READY | Prioridade:HIGH | TempoRestante:3 | I/O:NENHUM]
[t=000] Processo criado - Fila ALTA: [PID:3 | Status:READY | Prioridade:HIGH | TempoRestante:8 | I/O:FITA]
[t=000] Processo criado - Fila ALTA: [PID:4 | Status:READY | Prioridade:HIGH | TempoRestante:2 | I/O:NENHUM]
[t=000]
[t=000] INICIANDO SIMULAÇÃO - Round Robin com Feedback + I/O
[t=000] Quantum ALTA=2 | Quantum BAIXA=4 | Dispositivos: DISCO, FITA, IMPRESSORA
[t=000] Durações de I/O: DISCO=4 | FITA=6 | IMPRESSORA=3
[t=000]
[t=001] Troca de contexto - CPU para PID 1 (Fila: HIGH | Quantum: 2)
[t=001] Executando PID 1 | Resta: 4 | Quantum: 1/2 | I/O: DISCO
[t=001] PID 1 solicitou I/O [DISCO] | Bloqueado por 4 ticks
[t=002] Troca de contexto - CPU para PID 2 (Fila: HIGH | Quantum: 2)
[t=002] Executando PID 2 | Resta: 2 | Quantum: 1/2 | I/O: NENHUM
[t=003] Executando PID 2 | Resta: 1 | Quantum: 2/2 | I/O: NENHUM
[t=003] Quantum expirado, PID 2 REBAIXADO -> Fila BAIXA.
[t=004] Troca de contexto - CPU para PID 3 (Fila: HIGH | Quantum: 2)
[t=004] Executando PID 3 | Resta: 7 | Quantum: 1/2 | I/O: FITA
[t=004] PID 3 solicitou I/O [FITA] | Bloqueado por 6 ticks
[t=005] PID 1 concluiu I/O [DISCO] - retornou à Fila BAIXA
[t=005] Troca de contexto - CPU para PID 4 (Fila: HIGH | Quantum: 2)
[t=005] Executando PID 4 | Resta: 1 | Quantum: 1/2 | I/O: NENHUM
[t=006] Executando PID 4 | Resta: 0 | Quantum: 2/2 | I/O: NENHUM
[t=006] PID 4 FINALIZADO | Turnaround: 6 | Espera: 5 | Preempções: 0 | Eventos de I/O: 0
[t=007] Troca de contexto - CPU para PID 2 (Fila: LOW | Quantum: 4)
[t=007] Executando PID 2 | Resta: 0 | Quantum: 1/4 | I/O: NENHUM
[t=007] PID 2 FINALIZADO | Turnaround: 7 | Espera: 6 | Preempções: 1 | Eventos de I/O: 0
[t=008] Troca de contexto - CPU para PID 1 (Fila: LOW | Quantum: 4)
[t=008] Executando PID 1 | Resta: 3 | Quantum: 1/4 | I/O: DISCO
[t=009] Executando PID 1 | Resta: 2 | Quantum: 2/4 | I/O: DISCO
[t=010] PID 3 concluiu I/O [FITA] - retornou à Fila ALTA
[t=010] Executando PID 1 | Resta: 1 | Quantum: 3/4 | I/O: DISCO
[t=011] Executando PID 1 | Resta: 0 | Quantum: 4/4 | I/O: DISCO
[t=011] PID 1 FINALIZADO | Turnaround: 11 | Espera: 4 | Preempções: 0 | Eventos de I/O: 1
[t=012] Troca de contexto - CPU para PID 3 (Fila: HIGH | Quantum: 2)
[t=012] Executando PID 3 | Resta: 6 | Quantum: 1/2 | I/O: FITA
[t=013] Executando PID 3 | Resta: 5 | Quantum: 2/2 | I/O: FITA
[t=013] Quantum expirado, PID 3 REBAIXADO -> Fila BAIXA.
[t=014] Troca de contexto - CPU para PID 3 (Fila: LOW | Quantum: 4)
[t=014] Executando PID 3 | Resta: 4 | Quantum: 1/4 | I/O: FITA
[t=015] Executando PID 3 | Resta: 3 | Quantum: 2/4 | I/O: FITA
[t=016] Executando PID 3 | Resta: 2 | Quantum: 3/4 | I/O: FITA
[t=017] Executando PID 3 | Resta: 1 | Quantum: 4/4 | I/O: FITA
[t=017] Quantum expirado, PID 3 retornou -> Fila BAIXA.
[t=018] Troca de contexto - CPU para PID 3 (Fila: LOW | Quantum: 4)
[t=018] Executando PID 3 | Resta: 0 | Quantum: 1/4 | I/O: FITA
[t=018] PID 3 FINALIZADO | Turnaround: 18 | Espera: 8 | Preempções: 2 | Eventos de I/O: 1
[t=018]
[t=018] FIM DA SIMULAÇÃO
[t=018]
[t=018] MÉTRICAS FINAIS
[t=018] Tempo total da simulação : 18 ticks
[t=018] Processos finalizados : 4
[t=018] Total de preempções : 3
[t=018] Eventos de I/O - DISCO : 1
[t=018] Eventos de I/O - FITA : 1
[t=018] Eventos de I/O - IMPRESSORA : 0
[t=018] Turnaround médio : 10.50 ticks
[t=018] Tempo médio de espera : 5.75 ticks
[t=018] CPU ociosa : 0.6%
[t=018]
[t=018] PID | Turnaround | Espera | Preempções | Eventos I/O | Dispositivo
[t=018] -----
[t=018] 1 | 11 | 4 | 0 | 1 | DISCO
[t=018] 2 | 7 | 6 | 1 | 0 | NENHUM
[t=018] 3 | 18 | 8 | 2 | 1 | FITA
[t=018] 4 | 6 | 5 | 0 | 0 | NENHUM

Log salvo em: output/execution_log.txt
PS C:\Users\Laissa\OneDrive - BEG\Área de Trabalho\clone\SISOP-2026-01-G3>

```



Interpretação

A leitura do log permite acompanhar cada decisão do escalonador em tempo real. Alguns pontos relevantes observados na execução:

- **PID 1** utilizou apenas 1 tick de CPU antes de solicitar I/O de disco. Ficou bloqueado por 4 ticks, retornou à fila BAIXA e completou o restante do processamento nessa fila. Turnaround de 11 ticks, incluindo o tempo de I/O.
- **PID 2** consumiu o quantum completo da fila ALTA (2 ticks) sem finalizar nem solicitar I/O, sendo rebaixado para a fila BAIXA. Finalizou no primeiro tick da fila BAIXA. Única preempção do processo.
- **PID 3** foi o processo de maior carga: 8 ticks de CPU e 1 evento de I/O de fita (6 ticks). Após retornar da fita à fila ALTA, ainda precisou de 3 rodadas de CPU e sofreu 2 preempções adicionais antes de finalizar. Turnaround de 18 ticks.
- **PID 4** foi o mais simples: sem I/O, apenas 2 ticks de CPU, finalizado sem nenhuma preempção.
- A **CPU não ficou ociosa** em nenhum momento (0.0%), demonstrando boa utilização dos recursos com os processos definidos.

Conclusão

O trabalho permitiu consolidar conceitos fundamentais de escalonamento e gerenciamento de processos em Sistemas Operacionais.

Aprendizados:

- Funcionamento do Round Robin com Feedback;
- Impacto das operações de I/O no escalonamento;
- Importância de métricas como turnaround, tempo de espera e CPU ociosa;
- Uso de Git/GitHub para integração do trabalho em grupo;
- Execução padronizada com Docker.



Dificuldades:

- Sincronizar alterações entre branches;
- Resolver conflitos no Git/GitHub;
- Alinhar a integração entre os arquivos do projeto;
- Dockerizar e testar a execução em outro ambiente;
- Garantir que os logs e métricas refletissem corretamente o comportamento da simulação.

Melhorias futuras:

- Suporte a múltiplas solicitações de I/O por processo;
- Inclusão de novos algoritmos de escalonamento para comparação;
- Interface gráfica para visualização das filas e execução em tempo real.

